T3000 API & Protocol

Quick Start Overview · built Nov 13, 2026

Quick Start Overview

This guide will help you get started with T3000 Building Automation System quickly and efficiently.

What is T3000?

T3000 is a powerful building automation software platform designed to:

- **Monitor** - Track all building systems in real-time
- **Control** - Adjust setpoints and schedules remotely
- **Analyze** - Review historical data and trends
- **Optimize** - Improve energy efficiency and comfort

Installation Steps

1. **Download** - Get the latest version from the downloads page
2. **Install** - Run the installer and follow the wizard
3. **Configure** - Set up network and communication settings
4. **Connect** - Add your first device to start monitoring

Basic Workflow

TEXT

```
1. Connect Devices
   ↓
2. Configure Data Points
   ↓
3. Create Graphics/Dashboards
   ↓
4. Set Up Schedules
   ↓
5. Configure Alarms
```

First Steps

1. Launch T3000

After installation, launch the application from your Start menu or desktop shortcut.

2. Network Setup

Configure your network settings to communicate with devices:

- IP Address range
- Subnet mask
- Communication protocol (Modbus TCP/RTU, BACnet)

3. Add Your First Device

Navigate to **Device Management** and click **Add Device** to scan for and connect to devices on your network.

4. Explore the Interface

Familiarize yourself with the main sections:

- **Dashboard** - Overview of system status
- **Devices** - Manage connected controllers
- **Data Points** - View and control inputs, outputs, variables
- **Graphics** - Visual representations of your systems
- **Trends** - Historical data charts

Common Use Cases

HVAC Control: Monitor temperature, humidity, and control setpoints across multiple zones.

Energy Management: Track energy consumption and identify optimization opportunities.

Alarm Monitoring: Get notified of critical events in real-time.

Scheduling: Automate building operations based on occupancy and time of day.

Next Steps

- Installation Guide - Detailed installation instructions
- Configuration - Configure your system settings
- Device Management - Connect your first device

Getting Help

If you encounter any issues:

- Check the FAQ
- Review Troubleshooting Guide
- Contact support@temcocontrols.com

Quick Start Overview

System Architecture

T3000 Web is a modern building automation platform built on:

Frontend Stack:

- React/TypeScript for UI components
- Vite for build tooling
- WebSocket for real-time communication

Backend Stack:

- Rust API server (Port 9103)
- SQLite database for persistence
- BACnet/Modbus protocol handlers

Communication Protocols:

- BACnet/IP (UDP 47808)
- Modbus TCP (TCP 502)
- WebSocket API (WS 9103)
- REST API (HTTP 9103)

Quick Start for Developers

1. Installation via Package Manager

SHELL

```shell
## Download and extract
curl -O https://temcocontrols.com/downloads/t3000-web-v9.0.zip
unzip t3000-web-v9.0.zip -d /opt/t3000

## Run installer with silent mode
sudo ./T3000_Setup_v9.0.exe /S

## Or using Chocolatey (Windows)
choco install t3000-building-automation
```

2. Environment Configuration

Create configuration file `config.json`:

JSON

```json
{
  "network": {
    "adapter": "Ethernet",
    "ipRange": "192.168.1.0/24",
    "protocols": {
      "modbusTcp": { "enabled": true, "port": 502 },
      "bacnet": { "enabled": true, "port": 47808 },
      "http": { "enabled": true, "port": 9103 }
    }
  },
  "database": {
    "path": "./data/t3000.db",
    "backup": {
      "enabled": true,
      "interval": "daily",
      "retention": 30
    }
  }
}
```

3. Programmatic Device Discovery

TYPESCRIPT

```typescript
import { T3000Client } from '@temco/t3000-sdk';

const client = new T3000Client({
  host: 'localhost',
  port: 9103
});

// Discover devices
const devices = await client.discover({
  protocol: 'bacnet',
  timeout: 30000,
  subnet: '192.168.1.0/24'
});

console.log(`Found ${devices.length} devices`);

// Connect to device
const device = await client.connect({
  ip: '192.168.1.100',
  port: 47808,
  deviceId: 389001
});
```

4. API Integration Example

JAVASCRIPT

```javascript
// WebSocket connection for real-time data
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'OUT1']
  }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  console.log(`${data.point}: ${data.value}`);
};
```

5. System Initialization

PYTHON

```python
from t3000 import T3000System

## Initialize system
system = T3000System(config_path='config.json')

## Start services
system.start_api_server(port=9103)
system.start_bacnet_service()
system.start_modbus_service()

## Load devices from database
devices = system.load_devices()
print(f"Loaded {len(devices)} devices")

## Enable auto-discovery
system.enable_discovery(interval=300)  # 5 minutes
```

Integration Patterns

REST API Usage

SHELL

```shell
## Get device list
curl http://localhost:9103/api/devices

## Read data point
curl http://localhost:9103/api/devices/389001/points/IN1

## Write output value
curl -X PUT http://localhost:9103/api/devices/389001/points/OUT1 \
  -H "Content-Type: application/json" \
  -d '{"value": 75.0}'
```

Database Schema

SQL

```sql
-- Devices table
CREATE TABLE devices (
  id INTEGER PRIMARY KEY,
  serial_number INTEGER UNIQUE,
  ip_address TEXT,
  port INTEGER,
  protocol TEXT,
  status TEXT
);

-- Data points table
CREATE TABLE data_points (
  id INTEGER PRIMARY KEY,
  device_id INTEGER,
  point_type TEXT,
  label TEXT,
  value REAL,
  units TEXT,
  last_update INTEGER,
  FOREIGN KEY (device_id) REFERENCES devices(id)
);
```

Advanced Configuration

Custom Protocol Handler

RUST

```rust
use t3000_api::protocol::{ProtocolHandler, Device};

struct CustomProtocol;

impl ProtocolHandler for CustomProtocol {
    async fn discover(&self) -> Vec<Device> {
        // Custom discovery logic
        vec![]
    }

    async fn read_point(&self, device: &Device, point: &str) -> f64 {
        // Custom read logic
        0.0
    }
}

// Register handler
system.register_protocol("custom", CustomProtocol);
```

Next Steps

- REST API Reference
- WebSocket API
- Modbus Protocol